TECNM - INSTITUTO TECNOLÓGICO DE CULIACÁN

ASIGNATURA:
TÓPICOS DE INTELIGENCIA ARTIFICIAL

CARRERA: INGENIERÍA EN SISTEMAS COMPUTACIONALES

TRABAJO:
SISTEMA DE ENRUTAMIENTO PARA TIENDAS DE AUTOSERVICIO

TITULO:
OPTIMIZACIÓN DE RUTAS DE DISTRIBUCIÓN USANDO RECOCIDO SIMULADO

NOMBRE DEL MAESTRO:
ZURIEL DATHAN MORA FÉLIX

NOMBRE DEL ESTUDIANTE(S):
GALLEGOS RAMÍREZ HILARIO
RODRÍGUEZ MORENO AMÍLCAR

CULIACÁN SINALOA, 30 DE MARZO DEL 2026.

1. Índice

1. Índice 1
2. Introducción 2
3. Objetivos de la investigación 2
3.1 Objetivo general 2
3.2 Objetivos específicos 3
4. Justificación 4
5. Alcance del proyecto 4
5.1 Qué incluye 4
5.2 Qué no incluye 5
6. Desarrollo 5
6.1 Análisis del problema logístico 5
6.2 Modelado del problema como un caso de optimización combinatoria 7
6.3 Justificación del algoritmo heurístico 9
6.4 Implementación computacional 10
6.5 Evaluación del rendimiento de la solución 12
7. Agenda de Trabajo 17
8. Conclusiones 18

2. Introducción

El presente proyecto aborda un problema logístico de distribución de productos en una red compuesta por 100 nodos, donde 10 de ellos representan centros de distribución y los 90 restantes corresponden a tiendas ubicadas en un área aproximada de 10 kilómetros cuadrados. Cada centro de distribución debe abastecer a un conjunto de tiendas, lo que implica definir rutas eficientes que minimicen los costos asociados al transporte. Para ello, se cuenta con matrices de costos de distancia y combustible entre cada par de nodos, las cuales permiten modelar el problema como uno de optimización combinatoria, específicamente relacionado con el problema del agente viajero y sus variantes.

Dada la naturaleza combinatoria del problema, el número de posibles soluciones crece exponencialmente conforme aumenta el número de nodos, lo que hace inviable el uso de métodos exactos en tiempos razonables. En este contexto, el uso de métodos heurísticos se vuelve relevante, ya que permiten obtener soluciones cercanas al óptimo en tiempos computacionales eficientes. Este proyecto propone el uso de técnicas heurísticas para la asignación de tiendas a centros de distribución y la generación de rutas de abastecimiento, con el objetivo de mejorar la eficiencia operativa del sistema logístico.

Los informes y documentación de los avances se publicarán en el siguiente repositorio:
https://github.com/Amilcar-itc/Git-TopicosIA/tree/main/proyectosModuloII/SistemaEnrutamientoTiendasAutoservicio

3. Objetivos de la investigación

3.1 Objetivo general

Diseñar e implementar una solución computacional basada en algoritmos heurísticos (Recocido Simulado o Búsqueda Tabú) para optimizar las rutas de distribución de productos comestibles desde centros de distribución hacia sucursales de una cadena de restaurantes en Culiacán, Sinaloa, mejorando la eficiencia logística y reduciendo costos operativos.

El objetivo principal para esta tarea es diseñar e implementar una solución computacional que utilice algoritmos heurísticos, específicamente Recocido simulado o Búsqueda tabú. Esta solución debe tener como objetivo optimizar las rutas de distribución de productos de abarrotes desde los centros de distribución

hasta varias sucursales de una cadena de restaurantes ubicada en Culiacán, Sinaloa. Al hacer esto, ayudará a mejorar la eficiencia logística y reducir los costos operativos.

3.2 Objetivos específicos

1. Analizar el problema logístico

- Comience por comprender a fondo la distribución de productos comestibles en el área de estudio.
    - Investigar los desafíos logísticos actuales que enfrenta la cadena de restaurantes.
    - Identificar los factores clave que afectan la distribución, como los patrones de tráfico, los plazos de entrega y la perecibilidad del producto.

2. Modele el problema como un caso de optimización combinatoria

- Defina claramente los parámetros de su problema.
    - Crear un modelo matemático que represente las rutas de distribución como un problema de optimización combinatoria.
    - Considere cómo se pueden combinar las diferentes rutas y las implicaciones de cada combinación en la eficiencia y el costo.

3. Seleccionar y justificar el algoritmo heurístico

- Investigue algoritmos de búsqueda de recocido simulado y tabú.
    - Compare sus fortalezas y debilidades en el contexto de su problema específico.
    - Elija el algoritmo que crea que es más adecuado para optimizar las rutas de distribución y proporcione una justificación detallada de su elección.

4. Desarrollar una implementación computacional

- Usando un lenguaje de programación de su elección, comience a codificar el algoritmo heurístico seleccionado.
    - Asegúrese de que su código esté bien estructurado e incluya comentarios para explicar su lógica.
    - Pruebe su implementación con datos de muestra para asegurarse de que funciona correctamente.

5. Evaluar el rendimiento de la solución

- Después de ejecutar las simulaciones, evalúe el rendimiento de la solución.
    - Utilice métricas relevantes como la distancia total recorrida, el tiempo de entrega y los costos de combustible para evaluar la eficiencia.
    - Compare los resultados de su algoritmo con los datos de referencia para determinar las mejoras.

6. Documentar el proceso completo del proyecto

- Mantenga un registro detallado de cada paso que dé a lo largo del proyecto.
    - Incluya su análisis, modelado, selección de algoritmos, implementación, resultados de simulación y evaluación del rendimiento.
    - Prepare un informe final que resuma sus hallazgos e incluya los desafíos que enfrentó y cómo los superó.

4. Justificación

La optimización de rutas de distribución es un problema crítico en la toma de decisiones dentro de sistemas logísticos, ya que impacta directamente en costos operativos, tiempos de entrega y uso de recursos. En escenarios reales, como el abastecimiento de productos en cadenas comerciales, pequeñas mejoras en la planificación de rutas pueden traducirse en reducciones significativas de costos de combustible y en un mejor aprovechamiento del tiempo. Debido a la complejidad del problema, los métodos heurísticos representan una alternativa viable frente a los métodos exactos, ya que permiten explorar el espacio de soluciones de manera eficiente sin requerir un costo computacional elevado.

El uso de heurísticas facilita la adaptación del modelo a condiciones cambiantes, como variaciones en la demanda, restricciones de capacidad o modificaciones en la red de distribución. En este proyecto, la implementación de técnicas como la heurística de arrepentimiento para la asignación de nodos y métodos de búsqueda local para la optimización de rutas permite construir una solución flexible y escalable. Esto no solo contribuye a mejorar la calidad de las decisiones logísticas, sino que también proporciona una base sólida para la incorporación de algoritmos más avanzados, como recocido simulado o búsqueda tabú.

5. Alcance del proyecto

5.1 Qué incluye

- Modelado del problema de distribución como un problema de optimización combinatoria.
- Construcción de una matriz de costos a partir de datos de distancia y consumo de combustible.
- Implementación de un método heurístico para la asignación de tiendas a centros de distribución.
- Definición de áreas de distribución con restricciones de capacidad por centro.

- Generación de rutas de abastecimiento basadas en las áreas de distribución.
- Implementación de una función objetivo para evaluar el costo de cada ruta.
- Desarrollo de mecanismos de exploración de soluciones vecinas mediante técnicas de intercambio (swap).
- Implementación computacional en Python con estructura modular.
- Evaluación del desempeño de las soluciones generadas mediante métricas de costo.

5.2 Qué no incluye

- Consideración de variables dinámicas como tráfico en tiempo real o condiciones climáticas.
- Restricciones operativas avanzadas (ventanas de tiempo, múltiples vehículos por centro, etc.).
- Optimización exacta del problema mediante métodos deterministas.
- Integración con sistemas reales de geolocalización o bases de datos externas.
- Interfaz gráfica de usuario.
- Validación con datos reales de una empresa específica (solo simulación).

6. Desarrollo

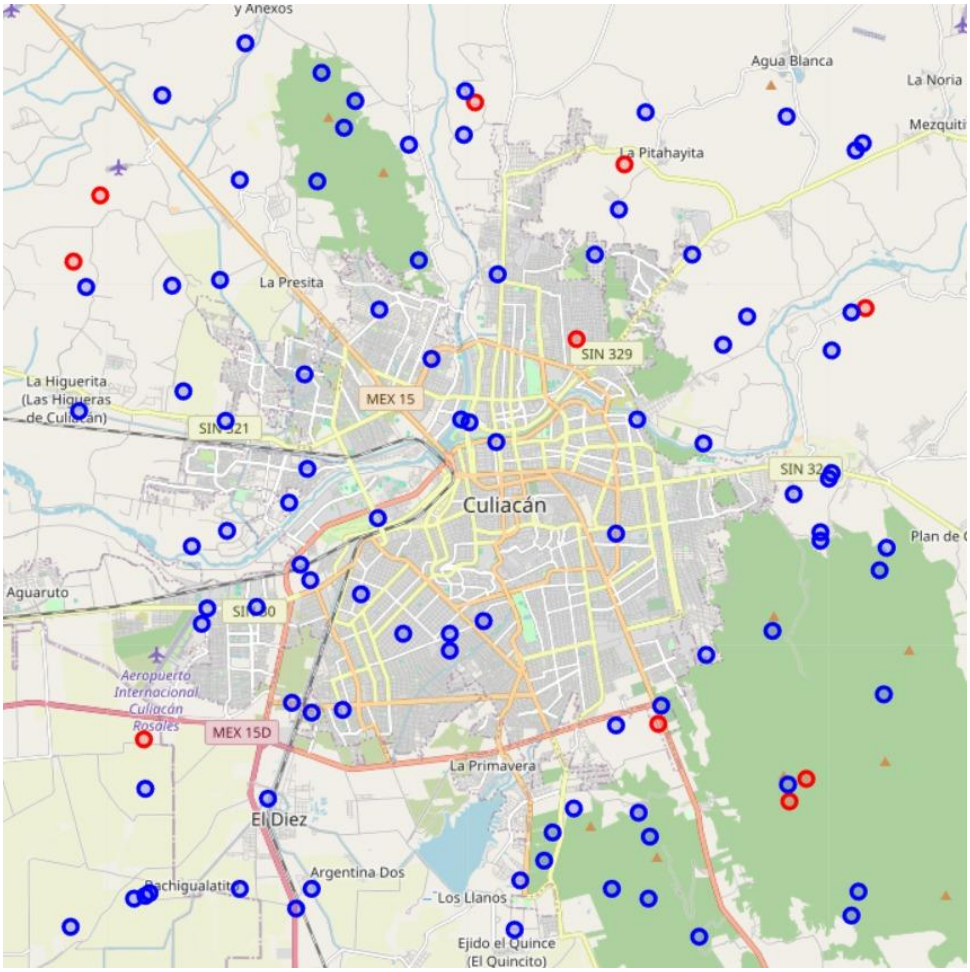
6.1 Análisis del problema logístico

La distribución de productos comestibles dentro de un área geográfica limitada representa un problema logístico complejo, especialmente cuando se busca optimizar tanto los costos operativos como los tiempos de entrega. En este caso de estudio, se considera una red de 100 nodos distribuidos en un área aproximada de 10 km², donde los primeros 10 nodos corresponden a centros de distribución y los 90 restantes representan tiendas que deben ser abastecidas.

Uno de los principales desafíos logísticos radica en la necesidad de coordinar múltiples rutas de distribución que parten de diferentes centros, garantizando que cada tienda sea atendida de manera eficiente. Este tipo de problema se complica debido a la interacción entre diversos factores que influyen directamente en la calidad de la solución.

Entre los factores clave que afectan la distribución se encuentran:

- Costos de transporte, determinados por la distancia recorrida y el consumo de combustible.
- Limitaciones operativas, como la capacidad máxima de tiendas que puede atender cada centro de distribución.
- Estructura geográfica del problema, que influye en la cercanía relativa entre nodos.
- Secuencia de visitas, ya que el orden en el que se recorren las tiendas impacta directamente en el costo total de la ruta.



Img.1 Visualización gráfica del problema simulado.

Aunque en este modelo no se consideran explícitamente variables como el tráfico en tiempo real o los productos perecederos, estos factores son relevantes en escenarios reales y justifican la necesidad de utilizar modelos flexibles que puedan adaptarse a condiciones dinámicas. En resumen, el problema se enfoca en diseñar rutas de distribución eficientes que minimicen el costo total de transporte,

asegurando al mismo tiempo que cada tienda sea asignada a un centro de distribución adecuado.

6.2 Modelado del problema como un caso de optimización combinatoria

El problema descrito puede modelarse como un caso de optimización combinatoria, en el cual se busca encontrar la mejor configuración posible entre un conjunto finito pero muy grande de soluciones. Donde se definen los siguientes parámetros:

N = un conjunto de 100 nodos del 0 al 99
cd = un subconjunto de N que representa a los centros de distribución
T = un subconjunto de N que representa a las tiendas
M = matriz de costos $n \times n$ donde cada elemento $M[i][j]$ representa el costo de trasladarse del nodo i al j

La matriz de costos se obtiene mediante la combinación de dos matrices iniciales: distancia y consumo de combustible, las cuales se multiplican elemento a elemento para obtener un costo compuesto.

Posteriormente se realiza la asignación de tiendas a centros (*clustering*), es decir, cada tienda debe ser asignada a un único centro de distribución, formando lo que se denomina un *área de distribución*. Esta asignación se realiza utilizando una heurística basada en prioridad:

$$p = Costo(cd2) - Costo(cd1)$$

Donde:

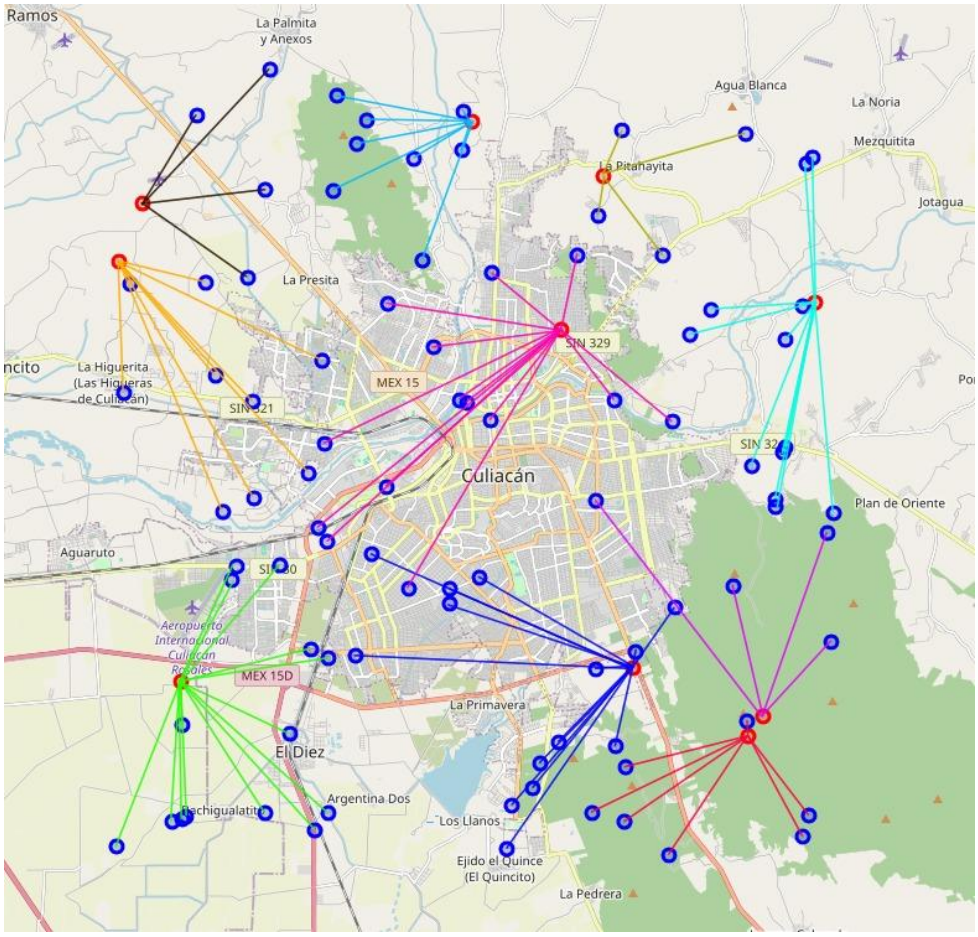
- $cd1$ es el Centro de Distribución más cercano
- $cd2$ es el segundo Centro de Distribución más cercano

Esta estrategia corresponde a una variante de la *regret heuristic*, ya que prioriza la asignación de nodos cuya segunda mejor opción es significativamente peor, reduciendo el riesgo de asignaciones subóptimas. Además, se establece una restricción de capacidad máxima de 14 tiendas por centro de distribución.

El resultado de este algoritmo fue el siguiente:

Centro de Distribución	Tiendas asignadas	Total
1	[12, 59, 37, 40, 55, 44, 48]	7
2	[62, 31, 56, 16, 91, 84, 11, 57, 79]	9
3	[36, 27, 89, 98]	4
4	[42, 69, 67, 47, 92, 93, 61, 45, 49, 94, 13, 85, 88, 53]	14
5	[76, 21, 75, 80, 81, 63, 96, 51, 23, 65, 60, 68]	12
6	[72, 19, 25, 86, 97, 26, 43, 20]	8
7	[17, 35, 66, 95, 90, 15, 58, 83, 41, 77, 100, 70, 24, 99]	14
8	[74, 34, 87, 64]	4
9	[29, 54, 73, 50, 46, 39, 52, 22, 78, 82, 28, 30, 32, 18]	14
10	[71, 33, 38, 14]	10

De manera gráfica, el algoritmo indica que los centros de distribución *cd* (en rojo) son responsables de las tiendas (en azul) generando las siguientes áreas de distribución:



Img.2 Representación gráfica de las áreas de distribución

Una vez definidas las áreas de distribución, el problema se reduce a múltiples instancias del problema del agente viajero (TSP), donde cada centro de distribución debe encontrar la ruta óptima para visitar todas sus tiendas asignadas y regresar al punto de origen.

6.3 Justificación del algoritmo heurístico

Dado que el problema pertenece a la clase de optimización combinatoria NP-difícil, se recurre al uso de algoritmos heurísticos para obtener soluciones aproximadas en tiempos razonables. Se consideran las siguientes alternativas de algoritmos de optimización:

1. Recocido simulado (Simulated Annealing)

El recocido simulado es un algoritmo probabilístico que permite escapar de óptimos locales mediante la aceptación controlada de soluciones peores. Su principal ventaja es la capacidad de explorar el espacio de soluciones de manera amplia, lo que incrementa la probabilidad de encontrar soluciones cercanas al óptimo global.

Ventajas:

- Evita quedar atrapado en óptimos locales
- Buen desempeño en problemas complejos
- Flexible y adaptable

Desventajas:

- Requiere ajuste de parámetros (temperatura, enfriamiento)
- Puede ser computacionalmente costoso

2. Búsqueda tabú (Tabu Search)

La búsqueda tabú es un método basado en memoria que evita regresar a soluciones previamente exploradas mediante el uso de una lista tabú. Este enfoque permite una exploración más estructurada del espacio de soluciones.

Ventajas:

- Evita ciclos
- Intensifica la búsqueda en regiones prometedoras
- Buen equilibrio entre exploración y explotación

Desventajas:

- Mayor complejidad de implementación
- Requiere definir estructuras de memoria adecuadas

Para este proyecto, se propone utilizar una estrategia inicial basada en búsqueda local con generación de vecinos mediante intercambio (swap), la cual puede extenderse posteriormente hacia un enfoque más robusto como el recocido simulado. La generación de vecinos se realiza intercambiando posiciones de tiendas dentro de una ruta, lo que permite explorar configuraciones cercanas de manera eficiente. Esta técnica presenta una complejidad computacional baja y es fácil de implementar, lo que la hace ideal como punto de partida.

Posteriormente, se puede incorporar recocido simulado para mejorar la calidad de las soluciones, permitiendo aceptar movimientos que incrementen temporalmente el costo con el fin de escapar de óptimos locales.

6.4 Implementación computacional

El proyecto se organiza en una carpeta principal que contiene un archivo principal de ejecución (main) y una carpeta de utilerías (Utilerias) donde se agrupan las funciones por responsabilidad

```
SistemaEnrutamientoTiendasAutoservicio/
│
├── MainEnrutamiento.py
└── Utilerias/
    ├── FuncionesDeRepresentacion.py
    ├── FuncionesDeMapeo.py
    └── FuncionesDeOptimizacion.py
```

Cada archivo tiene asignados funciones que complementan el algoritmo completo

- FuncionesDeRepresentacion.py

generar_matriz_costos()
Lee los archivos de entrada y construye la matriz de costos final mediante la multiplicación elemento a elemento.

imprimir_matriz_costos()
Muestra en consola la matriz de costos generada.

calcular_prioridades(costos, centros, tiendas)
Calcula la prioridad de cada tienda según la diferencia de costos entre sus dos centros más cercanos.

imprimir_prioridades()
Muestra las prioridades de todas las tiendas ordenadas de mayor a menor.

asignar_tiendas_a_centros(prioridades, centros, capacidad_max)
Asigna tiendas a centros de distribución respetando una capacidad máxima por centro.

imprimir_asignaciones()
Imprime las áreas de distribución generadas y las tiendas asignadas a cada centro.

generar_areas_de_distribucion()
Genera una estructura de datos simplificada con las áreas de distribución en formato (cd, tiendas).

validar_areas(areas)
Verifica que el número total de tiendas asignadas sea correcto.

imprimir_areas_de_distribucion()
Muestra las áreas de distribución utilizando la estructura generada.

- FuncionesDeMapeo.py

funcion_objetivo(matriz_de_costos, ruta)
Calcula el costo total de una ruta, sumando los costos desde el centro de distribución hacia las tiendas, entre tiendas consecutivas y el regreso al centro.

imprimir_ruta(matriz_de_costos, ruta)
Muestra en consola una ruta junto con su costo total, facilitando la visualización de resultados.

generar_swaps(ruta)
Genera rutas vecinas a partir de una ruta dada, intercambiando posiciones de tiendas (swaps) para explorar nuevas soluciones.

rutas_iguales(ruta_1, ruta_2)
Compara dos rutas y determina si son exactamente iguales.

- FuncionesDeOptimizacion.py

aceptar_solucion(delta, temperatura)
Decide si una solución vecina se acepta, permitiendo aceptar soluciones peores con cierta probabilidad.

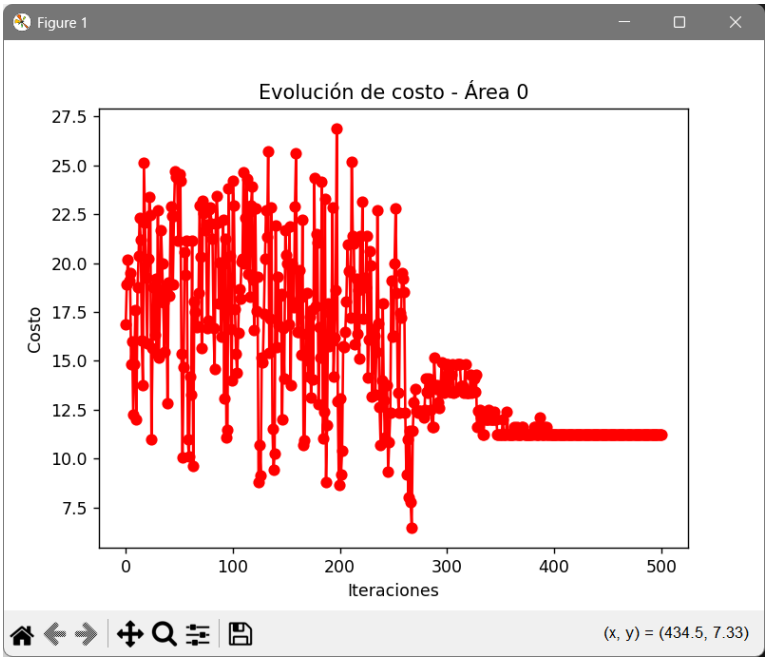
enfriar(temperatura, alpha)
Reduce la temperatura según un factor de enfriamiento.

recosido_simulado(...)
Aplica el algoritmo de recocido simulado para mejorar una ruta, generando soluciones vecinas y buscando minimizar el costo total.

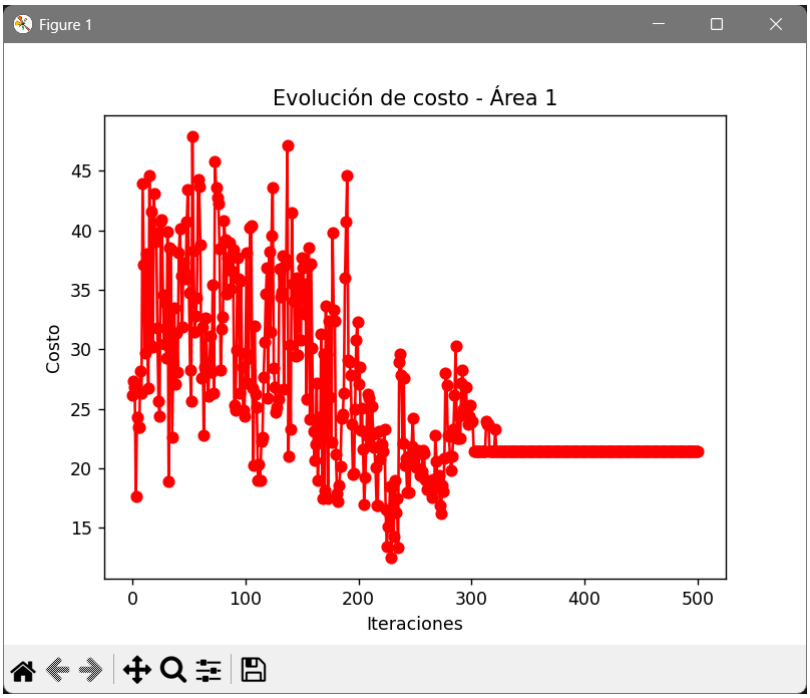
6.5 Evaluación del rendimiento de la solución

Como se estableció, se divide el conjunto N en 10 subconjuntos cd, es decir, se realizan 10 simulaciones, una por cada área de distribución, representadas desde Área 0 hasta Área 9. Los resultados son los siguientes:

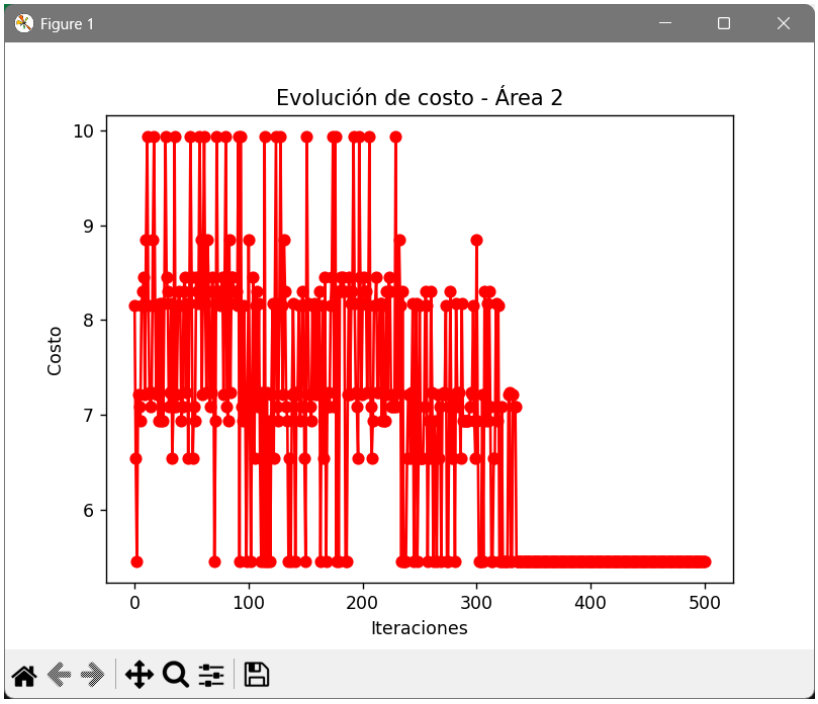
(Ordenados por su gráfica y posteriormente su inicio y fin de ruta y costo)



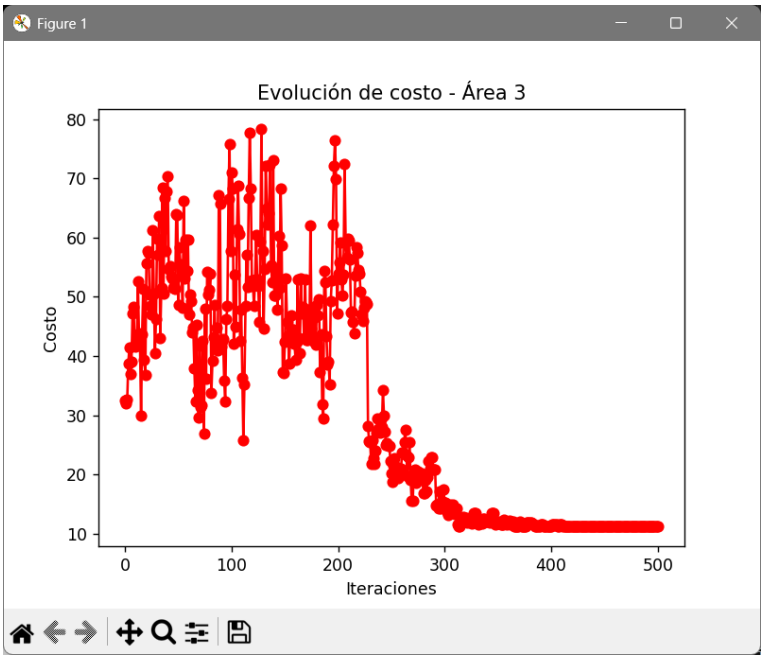
Ruta inicial: [11, 58, 36, 39, 54, 43, 47] -> Costo total: 16.8869
Ruta optimizada: [47, 54, 39, 58, 43, 11, 36] -> Costo total: 6.4549



Ruta inicial: [61, 30, 55, 15, 90, 83, 10, 56, 78] -> Costo total: 26.1780
Ruta optimizada: [78, 56, 61, 55, 30, 90, 83, 15, 10] -> Costo total: 12.3351

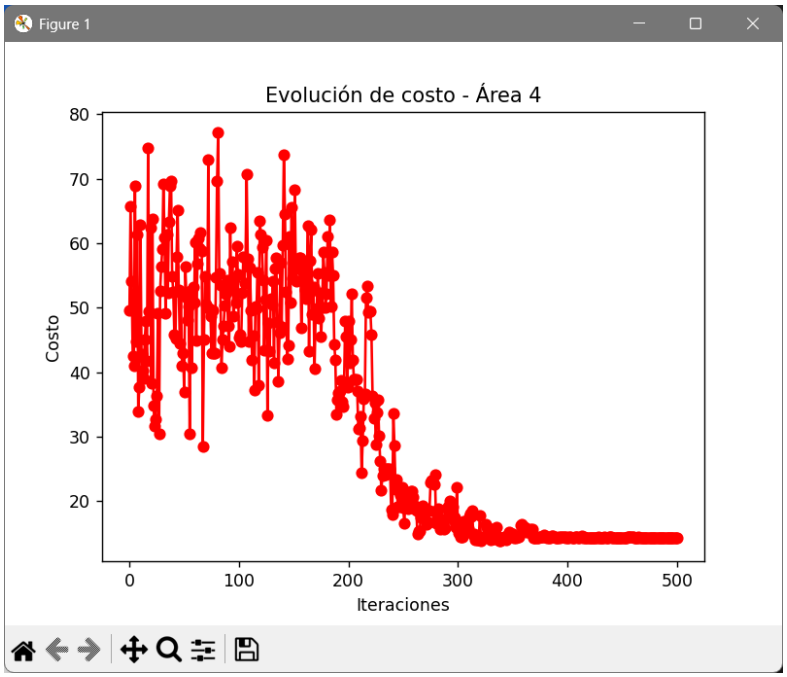


Ruta inicial: [35, 26, 88, 97] -> Costo total: 8.1579
Ruta optimizada: [35, 88, 97, 26] -> Costo total: 5.4515



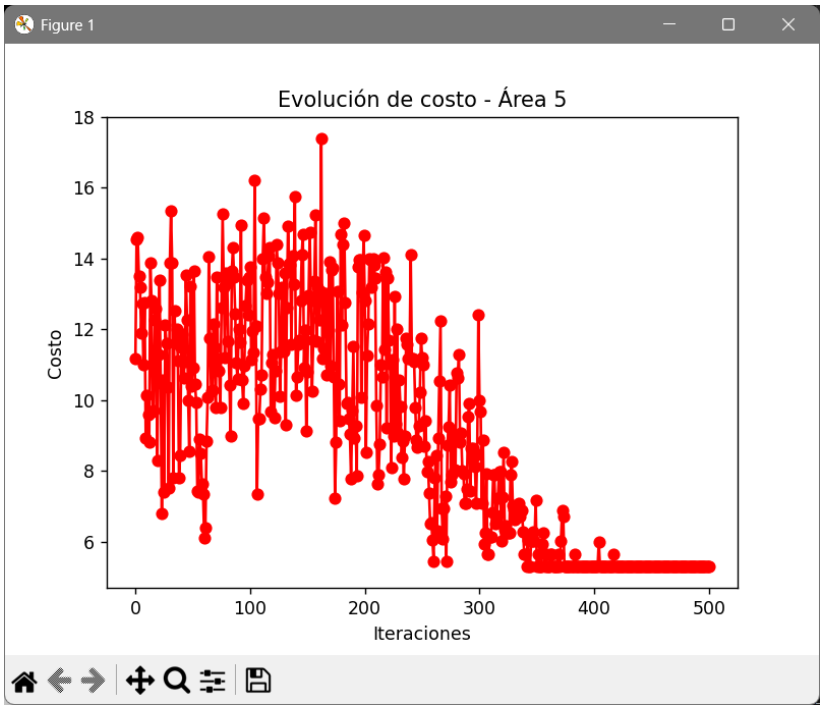
Ruta inicial: [41, 68, 66, 46, 91, 92, 60, 44, 48, 93, 12, 84, 87, 52] -> Costo total: 32.5480

Ruta optimizada: [46, 92, 87, 12, 66, 41, 68, 91, 60, 52, 84, 48, 93, 44] -> Costo total: 11.2331

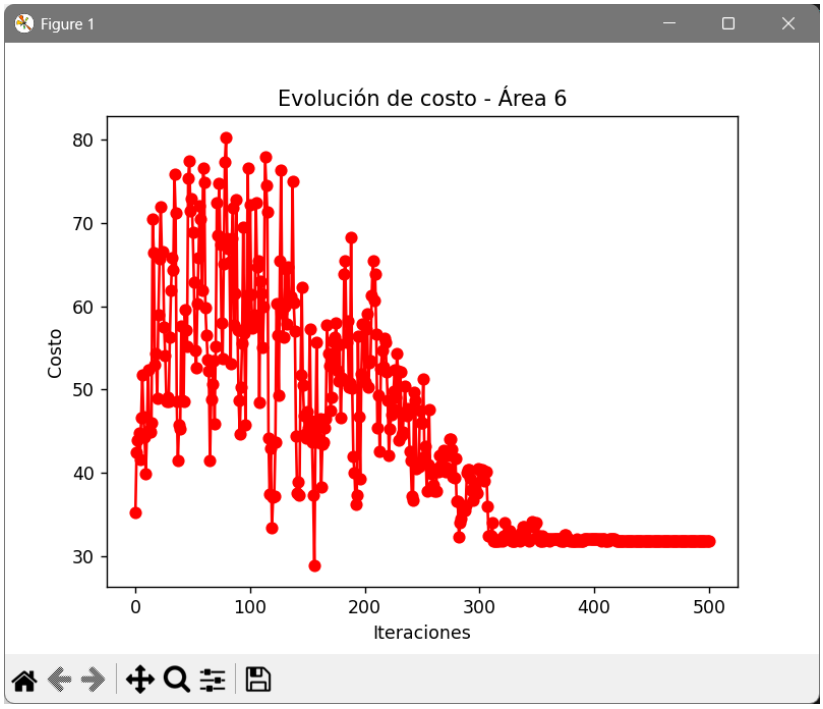


Ruta inicial: [75, 20, 74, 79, 80, 62, 95, 50, 22, 64, 59, 67] -> Costo total: 49.6121

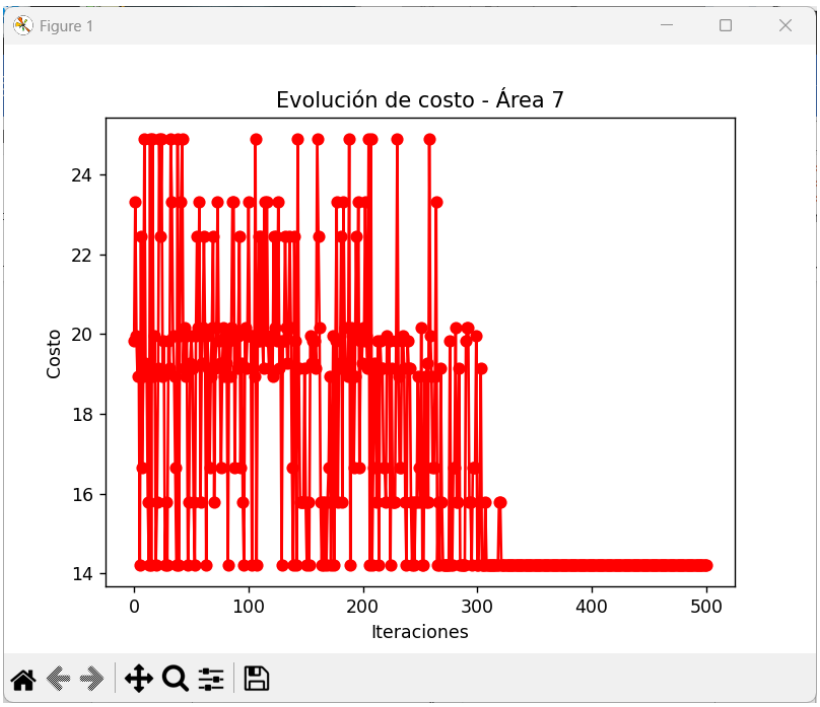
Ruta optimizada: [75, 67, 74, 64, 59, 22, 62, 79, 20, 50, 95, 80] -> Costo total: 13.8847



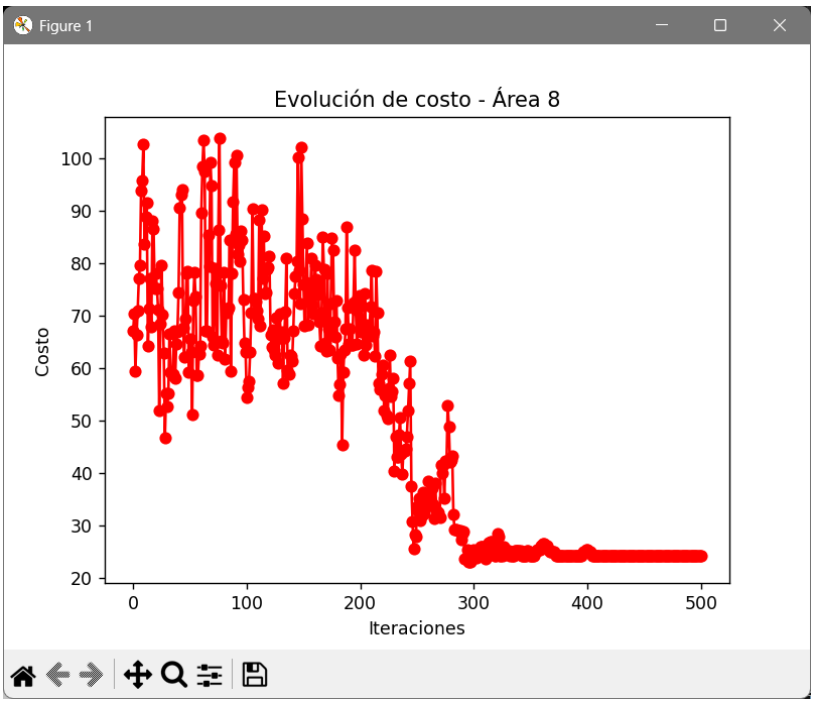
Ruta inicial: [71, 18, 24, 85, 96, 25, 42, 19] -> Costo total: 11.1814
Ruta optimizada: [96, 25, 19, 42, 18, 85, 71, 24] -> Costo total: 5.0805



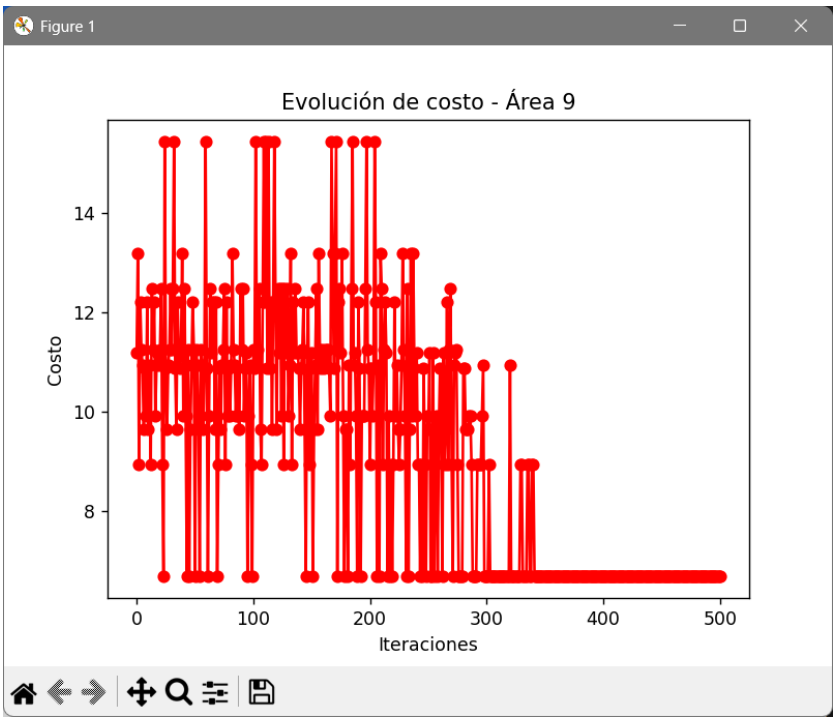
Ruta inicial: [16, 34, 65, 94, 89, 14, 57, 82, 40, 76, 99, 69, 23, 98] -> Costo total: 35.2842
Ruta optimizada: [40, 98, 23, 69, 14, 94, 34, 65, 89, 16, 57, 99, 76, 82] -> Costo total: 26.9885



Ruta inicial: [73, 33, 86, 63] -> Costo total: 19.8380
Ruta optimizada: [33, 73, 86, 63] -> Costo total: 14.2138



Ruta inicial: [28, 53, 72, 49, 45, 38, 51, 21, 77, 81, 27, 29, 31, 17] -> Costo total: 67.1751
Ruta optimizada: [17, 77, 27, 51, 49, 38, 45, 29, 72, 53, 31, 21, 28, 81] -> Costo total: 23.1478



Ruta inicial: [70, 32, 37, 13] -> Costo total: 11.1787
Ruta optimizada: [70, 37, 32, 13] -> Costo total: 6.6783

En resumen

Área de Distribución	Costo inicial	Costo Final	% de mejora
0	16.88	6.45	62%
1	26.17	12.33	53%
2	8.15	5.45	33%
3	32.54	11.23	65%
4	49.61	13.88	72%
5	11.18	5.08	55%
6	35.28	26.98	24%
7	19.83	14.21	28%
8	67.17	23.14	66%
9	11.17	6.67	40%

7. Agenda de Trabajo

El proyecto se desarrolla en un plazo de 18 días:

Fechas	Objetivo
Viernes 13 – Domingo 15	• Analizar el proyecto • Diseñar bases del documento

	• Capacitación en Git y Python
Lunes 16 – Jueves 19	• Generar estructura del repositorio • Investigar modelos de implementación • Modelado del problema
Viernes 20 – Domingo 22	• Diseño de modularización • Implementación de algoritmos • Representaciones gráficas
Lunes 23 – Jueves 26	• Unificación de algoritmos modularizados • Simulación y resultados
Viernes 27 – Lunes 30	• Corrección de errores • Documentación de investigación y resultados

8. Conclusiones

El método propuesto permite transformar un problema complejo de ruteo en una serie de subproblemas más manejables, mediante la combinación de una adecuada representación del sistema de costos y una estrategia de asignación basada en prioridades. La construcción de la matriz de costos a partir de variables relevantes como distancia y consumo de combustible proporciona una base realista para la toma de decisiones, mientras que la generación de áreas de distribución reduce significativamente la complejidad del problema global. La aplicación de una heurística de tipo “regret” para la asignación de tiendas introduce un criterio eficiente para priorizar decisiones críticas, lo cual se traduce en una mejor distribución inicial de los nodos. Posteriormente, el uso de un algoritmo de recocido simulado permite refinar las rutas dentro de cada área, explorando el espacio de soluciones de manera probabilística y evitando quedar atrapado en óptimos locales.

A lo largo del desarrollo se obtuvieron aprendizajes importantes relacionados con la importancia de la modularización del sistema, la correcta definición de estructuras de datos y la influencia de los parámetros en los algoritmos de optimización. Se observó que decisiones como la forma de generar vecinos o la velocidad de enfriamiento impactan directamente en la calidad de las soluciones obtenidas. Como trabajo futuro, se podrían incorporar métodos de optimización más robustos, con heurísticas para mejorar la generación de vecinos, así como estrategias adaptativas para el ajuste dinámico de parámetros del recocido simulado. También sería posible integrar restricciones más realistas, como capacidades variables o ventanas de tiempo, con el fin de acercar el modelo a escenarios reales de logística y distribución.